

Breakout

Realization

Nikola Panayotov

Fontys University of Applied Sciences – ICT

Table of Contents

Table of Contents	2
2 Tools and Technologies Used	3
2.1 How to build and run.....	3
3 Code Structure and Design Alignment	3
3.1 Project file structure	4
3.2 Selected code excerpts	4

1 Introduction

This document is the Realize evidence for the Breakout project: it describes how the design was turned into a working Java/LibGDX application, based on a review of the actual source code in the project repository (core module, package io.github.breaking_bricks), available on GitHub at <https://github.com/N1ki2K/breakout>.

2 Tools and Technologies Used

Tool / technology	Role
Java 17	Implementation language, as decided in Breakout Advice.
LibGDX 1.14.2	Game framework: rendering, input, fonts (FreeType), and the Game/Screen application lifecycle.
LWJGL3 3.4.3	Desktop backend used by the lwjgl3 module to launch the game as a Windows/desktop application.
Gradle (wrapper included)	Build system. The project was generated with gdx-liftoff, giving a core module (shared game logic) and an lwjgl3 module (desktop launcher).

2.1 How to build and run

- Run gradlew.bat lwjgl3:run (or ./gradlew lwjgl3:run on Linux) to launch the game directly.
- Run ./gradlew lwjgl3:jar to build a runnable JAR, found under lwjgl3/build/libs.
- Source code and full project: <https://github.com/N1ki2K/breakout>

3 Code Structure and Design Alignment

The table below checks the realized classes against the UML class diagram in Breakout Design.

Class	Matches design?	Notes
BreakoutGame	Yes	Extends LibGDX's Game, holds the active screen, and exposes a changeScreen() helper that disposes the previous screen.
GameConfig	Yes	Holds shared constants as designed (world size, wall positions). Also adds MISS_Y, used to detect when the ball is missed.
Paddle	Yes	Matches the design's fields and methods (moveLeft/moveRight, getBounds). Also tracks its own velocityX, used by Ball.bounceFromPaddle() for the hit-position/velocity "flick" effect described in the design (requirement S01).
Ball	Yes	Matches the design. bounceFromPaddle(hitPosition, paddleVelocity) implements the flick effect exactly as planned: horizontal direction is influenced by both where the ball hit the paddle and the paddle's velocity, then clamped to a maximum speed.

Class	Matches design?	Notes
Brick	Extended	Matches the design's fields and methods, but hit() now supports multi-hit "armoured" bricks (see section 3) rather than always destroying on the first hit.
StartScreen, GameScreen, GameOverScreen, WinScreen, PauseScreen	Yes	All implement LibGDX's Screen interface as designed. PauseScreen (added between the prototype and final design) is present and implemented.

3.1 Project file structure

The tree below shows the full layout of the repository. The.gdx-liftoff generator produced the two-module split: core holds all game logic and is backend independent, while lwjgl3 only contains the desktop entry point.

breakout/	
assets/	Runtime assets loaded by the game
fonts/	Inter TTF faces rendered via FreeType
Inter_18pt-Regular.ttf	
Inter_28pt-Regular.ttf	
textures/	
Bricks/ (png + svg)	brick1-5; types 4 and 5 are the armored bricks
ball/ (png + svg)	
paddle/ (png + svg)	
screen/ (svg)	Wireframe/screen mock-ups
screen_bg/ (png + svg)	start, game, win and game-over backgrounds
core/	Shared game logic (backend independent)
build.gradle	
src/main/java/io/github/breaking_bricks/	
BreakoutGame.java	Game subclass; changeScreen() + disposal
GameConfig.java	Shared constants (world size, walls, MISS_Y)
objects/	
Ball.java	Movement, wall bounce, bounceFromPaddle()
Brick.java	hit(), multi-hit armored logic, getBounds()
Paddle.java	moveLeft()/moveRight(), velocityX, getBounds()
screens/	
StartScreen.java	
GameScreen.java	Game loop, collisions, HUD, createBricks()
PauseScreen.java	
WinScreen.java	
GameOverScreen.java	
lwjgl3/	Desktop launcher (LWJGL3 backend)
build.gradle	
nativeimage.gradle	
icons/	logo.icns, logo.ico, logo.png
src/main/	
java/io/github/breaking_bricks/lwjgl3/	
Lwjgl3Launcher.java	main() entry point; window configuration
StartupHelper.java	macOS -XstartOnFirstThread handling
resources/	libgdx16/32/64/128.png window icons
docs/	Portfolio documents (Analysis, Advice, Design)
gradle/	
gradle-daemon-jvm.properties	
wrapper/	gradle-wrapper.jar + .properties
build.gradle	Root build script
settings.gradle	Declares the core and lwjgl3 modules
gradle.properties	
gradlew	Gradle wrapper (Linux/macOS)
gradlew.bat	Gradle wrapper (Windows)
README.md	

3.2 Selected code excerpts

The three snippets below were chosen because each one is where the realized behavior is most likely to diverge from the design, so seeing the actual code is more convincing than the summary table above. bounceFromPaddle() shows the hit-position/velocity "flick" calculation described in the design; hit() shows the multi-hit armored-brick logic that goes beyond the original design; and changeScreen() shows the screen-disposal helper used by every screen transition in the game.

Ball.bounceFromPaddle() - implements the hit-position and paddle-velocity "flick" effect from the design (io/github/breaking_bricks/objects/Ball.java):

```
public void bounceFromPaddle(
    float hitPosition,
    float paddleVelocity
) {
    dy = Math.abs(dy);

    dx += hitPosition * 250;
    dx += paddleVelocity * 0.15f;

    dx = MathUtils.clamp(dx, -500, 500);
}
```

Brick.hit() - the armored-brick logic that goes beyond the original design (io/github/breaking_bricks/objects/Brick.java):

```
public void hit() {
    switch (textureType) {
        case 5:
            textureType = 4;
            break;
        case 4:
            textureType = 3;
            break;
        default:
            destroyed = true;
            break;
    }
}
```

BreakoutGame.changeScreen() - the disposal wrapper mentioned in the table above (io/github/breaking_bricks/BreakoutGame.java):

```
private void changeScreen(Screen newScreen) {
    Screen oldScreen = getScreen();

    setScreen(newScreen);

    if (oldScreen != null) {
        oldScreen.dispose();
    }
}
```